PYTHON PROGRAMMING

CASE STUDY RECORD

Problem Statements • Assumptions • Algorithms • Mermaid Flowcharts • Pseudocode • Source Code • Outputs • Screenshots

Student Name	Chebrolu Jhansi Gowri
Branch	CSE
Roll Number	26000080
Section	2
Course	Python Programming

Prepared from the 10 Python source files supplied for this record. The source code is reproduced as supplied; sample outputs and console-style screenshots use the input values shown.

Contents

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Employee Payroll and Deduction Analysis
4. Retail Billing with Discount Rules
5. Campus Event Financial Planning
6. Examination Result and Grace-Mark Validation
7. Attendance Recovery Computation
8. ATM Transaction Sequence Simulation
9. Warehouse Inventory Reconciliation
10. Advanced Numeric Pattern Generator

Note on scope

The case-study titles in this record are aligned to the ten uploaded Python files. Where a requested advanced feature (such as applicant ranking, monthly anomaly detection, payroll overtime, itemized tax, sponsorship analysis, multi-subject results, withdrawal limits, stock reconciliation, or a hollow diamond) is not present in the supplied code, the assumption section explicitly states that limitation. The source code is reproduced as supplied and is not silently expanded or changed.

1. University Scholarship Allocation

Problem Statement

Write a Python program to check scholarship eligibility using aggregate marks, attendance percentage, and annual family income.

Assumptions

• Aggregate marks and attendance are entered as whole-number percentages.

• Eligibility requires marks of at least 75, attendance of at least 75%, and annual family income not greater than ₹2,00,000.

• The supplied program performs an eligibility check only; it does not rank multiple applicants or resolve ties.

Algorithm

Start.

Read aggregate marks, attendance percentage, and annual family income.

If marks ≥ 75, attendance ≥ 75, and income ≤ 200000, display eligible.

Otherwise, display not eligible.

Stop.

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input marks, attendance, income/]
    B --> C{marks >= 75 AND attendance >= 75 AND income <= 200000?}
    C -- Yes --> D[Display eligible]
    C -- No --> E[Display not eligible]
    D --> F([Stop])
    E --> F
```

Pseudocode

```
BEGIN
  INPUT marks, attendance, income
  IF marks >= 75 AND attendance >= 75 AND income <= 200000 THEN
      PRINT "Eligible for scholarship"
  ELSE
      PRINT "Not eligible for scholarship"
  END IF
END
```

1. University Scholarship Allocation — Implementation

Python Source Code (as supplied)

```
marks = int(input("Enter aggregate marks: "))
attendance = int(input("Enter attendance percentage: "))
income = int(input("Enter annual family income: "))

if marks >= 75 and attendance >= 75 and income <= 200000:
    print("Eligible for scholarship")
else:
    print("Not eligible for scholarship")
```

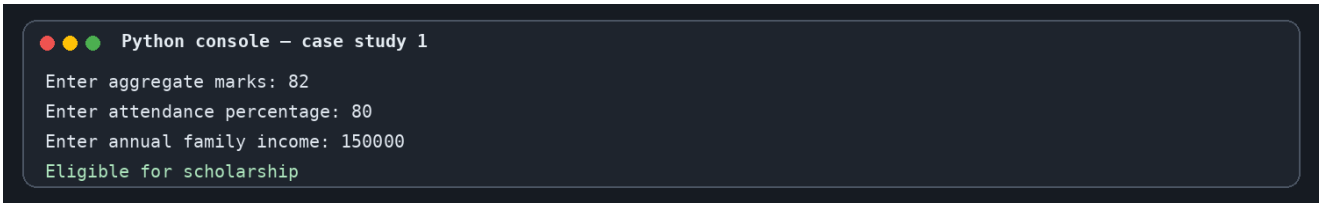
Sample Output / Console Transcript

Input values: 82, 80, 150000

```
Enter aggregate marks: 82
Enter attendance percentage: 80
Enter annual family income: 150000
Eligible for scholarship
```

The inputs meet all three conditions, so the program reports scholarship eligibility.

Output Screenshot



2. Progressive Electricity Billing

Problem Statement

Write a Python program to calculate an electricity bill using three progressive consumption slabs.

Assumptions

• Electricity consumption is entered as a non-negative whole number of units.

• The first 100 units cost ₹2 per unit, the next 100 units cost ₹3 per unit, and units above 200 cost ₹5 per unit.

• The supplied program does not include fixed charges, surcharges, rebates, or month-to-month anomaly detection.

Algorithm

Start.

Read electricity units.

If units ≤ 100, calculate all units at ₹2 each.

Else if units ≤ 200, calculate the first 100 at ₹2 and the remainder at ₹3.

Otherwise, calculate the first 100 at ₹2, the next 100 at ₹3, and the remaining units at ₹5.

Display the bill and stop.

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input units/]
    B --> C{units <= 100?}
    C -- Yes --> D[bill = units * 2]
    C -- No --> E{units <= 200?}
    E -- Yes --> F[bill = 200 + (units - 100) * 3]
    E -- No --> G[bill = 500 + (units - 200) * 5]
    D --> H[/Print bill/]
    F --> H
    G --> H
    H --> I([Stop])
```

Pseudocode

```
BEGIN
  INPUT units
  IF units <= 100 THEN
      bill = units * 2
  ELSE IF units <= 200 THEN
      bill = 100 * 2 + (units - 100) * 3
  ELSE
      bill = 100 * 2 + 100 * 3 + (units - 200) * 5
  END IF
  PRINT bill
END
```

2. Progressive Electricity Billing — Implementation

Python Source Code (as supplied)

```
units = int(input("Enter electricity units: "))

if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5

print("Electricity bill:", bill)
```

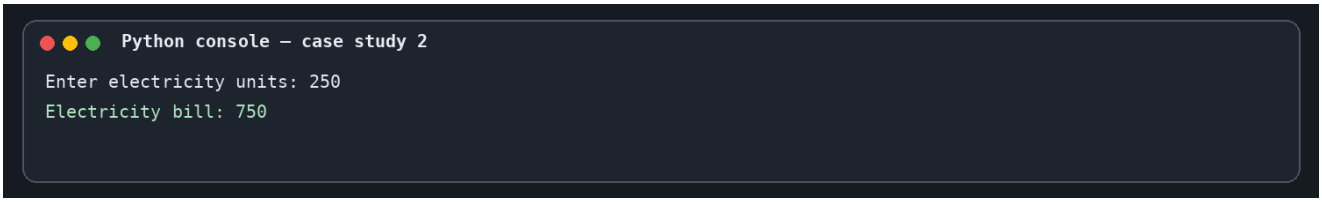
Sample Output / Console Transcript

Input values: 250

```
Enter electricity units: 250
Electricity bill: 750
```

For 250 units, the first 100 cost ₹200, the next 100 cost ₹300, and the remaining 50 cost ₹250. Total: ₹750.

Output Screenshot



3. Employee Payroll and Deduction Analysis

Problem Statement

Write a Python program to calculate HRA, DA, a deduction, and net salary from the basic salary.

Assumptions

• Basic salary is entered as a numeric amount in rupees.

• HRA is 10% of basic salary, DA is 5%, and the deduction is 8%.

• The supplied program does not calculate overtime, incentives, or progressive tax deductions.

Algorithm

Start.

Read basic salary.

Calculate HRA = 10% of basic salary.

Calculate DA = 5% of basic salary.

Calculate deduction = 8% of basic salary.

Calculate net salary = basic salary + HRA + DA − deduction.

Display all amounts and stop.

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input basic salary/]
    B --> C[Calculate HRA = 10%]
    C --> D[Calculate DA = 5%]
    D --> E[Calculate deduction = 8%]
    E --> F[Calculate net salary]
    F --> G[/Display salary details/]
    G --> H([Stop])
```

Pseudocode

```
BEGIN
  INPUT salary
  hra = salary * 0.10
  da = salary * 0.05
  deduction = salary * 0.08
  net_salary = salary + hra + da - deduction
  PRINT hra, da, deduction, net_salary
END
```

3. Employee Payroll and Deduction Analysis — Implementation

Python Source Code (as supplied)

```
salary = float(input("Enter basic salary: "))

hra = salary * 0.10
da = salary * 0.05
deduction = salary * 0.08
net_salary = salary + hra + da - deduction

print("HRA:", hra)
print("DA:", da)
print("Deduction:", deduction)
print("Net salary:", net_salary)
```

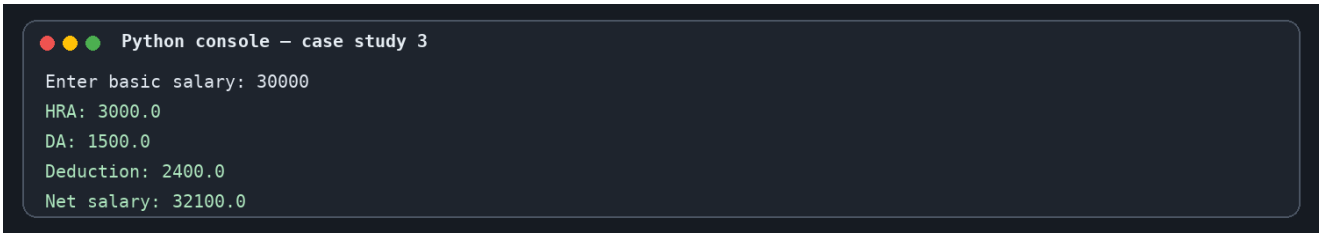
Sample Output / Console Transcript

Input values: 30000

```
Enter basic salary: 30000
HRA: 3000.0
DA: 1500.0
Deduction: 2400.0
Net salary: 32100.0
```

For a basic salary of ₹30,000, HRA is ₹3,000, DA is ₹1,500, deduction is ₹2,400, and net salary is ₹32,100.

Output Screenshot



4. Retail Billing with Discount Rules

Problem Statement

Write a Python program to calculate a discount and final bill based on purchase amount thresholds.

Assumptions

• Purchase amount is entered as a non-negative numeric value in rupees.

• Purchases of ₹5,000 or more receive 20% discount; purchases from ₹2,000 up to but not including ₹5,000 receive 10%; smaller purchases receive no discount.

• The supplied program applies one threshold discount to the total amount; it does not itemize products, apply category taxes, or cap concessions.

Algorithm

Start.

Read purchase amount.

If amount ≥ 5000, set discount to 20% of amount.

Else if amount ≥ 2000, set discount to 10% of amount.

Otherwise, set discount to zero.

Calculate total = amount − discount.

Display discount and final bill; stop.

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input purchase amount/]
    B --> C{amount >= 5000?}
    C -- Yes --> D[discount = 20%]
    C -- No --> E{amount >= 2000?}
    E -- Yes --> F[discount = 10%]
    E -- No --> G[discount = 0]
    D --> H[Calculate final bill]
    F --> H
    G --> H
    H --> I[/Print discount and total/]
    I --> J([Stop])
```

Pseudocode

```
BEGIN
  INPUT amount
  IF amount >= 5000 THEN
      discount = amount * 0.20
  ELSE IF amount >= 2000 THEN
      discount = amount * 0.10
  ELSE
      discount = 0
  END IF
  total = amount - discount
  PRINT discount, total
END
```

4. Retail Billing with Discount Rules — Implementation

Python Source Code (as supplied)

```
amount = float(input("Enter purchase amount: "))

if amount >= 5000:
    discount = amount * 0.20
elif amount >= 2000:
    discount = amount * 0.10
else:
    discount = 0

total = amount - discount

print("Discount:", discount)
print("Final bill:", total)
```

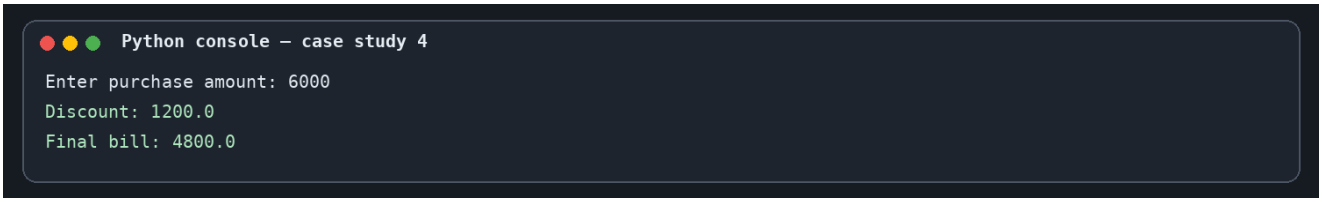
Sample Output / Console Transcript

Input values: 6000

```
Enter purchase amount: 6000
Discount: 1200.0
Final bill: 4800.0
```

A ₹6,000 purchase receives a 20% discount of ₹1,200, leaving a final bill of ₹4,800.

Output Screenshot



5. Campus Event Financial Planning

Problem Statement

Write a Python program to calculate event income, expenses, balance, and whether the event is within budget.

Assumptions

• Participant count is a whole number; fee and expenses are numeric amounts in rupees.

• Total income equals participants multiplied by fee per participant.

• The supplied program treats a zero or positive balance as within budget and a negative balance as a loss; it does not model sponsorships or break-even registration counts.

Algorithm

Start.

Read participant count, fee per participant, and total expenses.

Calculate income = participants × fee.

Calculate balance = income − expenses.

Display income, expenses, and balance.

If balance ≥ 0, display within budget; otherwise, display loss.

Stop.

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input participants, fee, expenses/]
    B --> C[Calculate income = participants * fee]
    C --> D[Calculate balance = income - expenses]
    D --> E[/Print income, expenses, balance/]
    E --> F{balance >= 0?}
    F -- Yes --> G[Event is within budget]
    F -- No --> H[Event has a loss]
    G --> I([Stop])
    H --> I
```

Pseudocode

```
BEGIN
  INPUT participants, fee, expenses
  income = participants * fee
  balance = income - expenses
  PRINT income, expenses, balance
  IF balance >= 0 THEN
      PRINT "Event is within budget"
  ELSE
      PRINT "Event has a loss"
  END IF
END
```

5. Campus Event Financial Planning — Implementation

Python Source Code (as supplied)

```
participants = int(input("Enter number of participants: "))
fee = float(input("Enter fee per participant: "))
expenses = float(input("Enter total expenses: "))

income = participants * fee
balance = income - expenses

print("Total income:", income)
print("Total expenses:", expenses)
print("Balance:", balance)

if balance >= 0:
    print("Event is within budget")
else:
    print("Event has a loss")
```

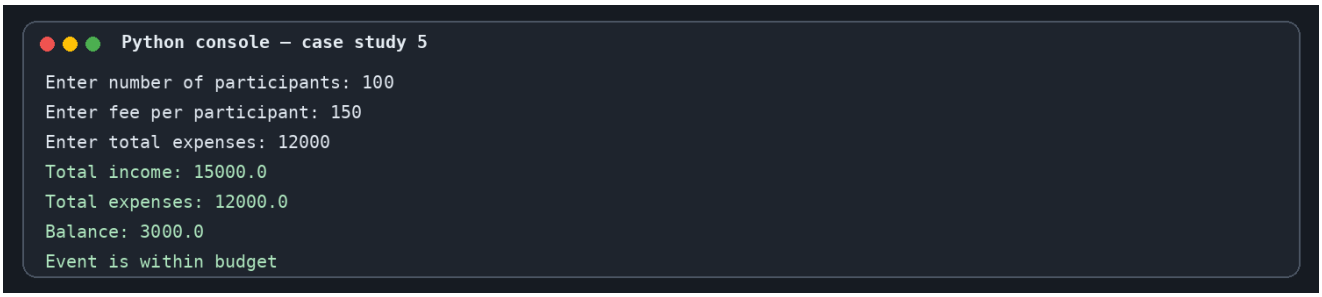
Sample Output / Console Transcript

Input values: 100, 150, 12000

```
Enter number of participants: 100
Enter fee per participant: 150
Enter total expenses: 12000
Total income: 15000.0
Total expenses: 12000.0
Balance: 3000.0
Event is within budget
```

Income is ₹15,000 and expenses are ₹12,000, so the positive balance is ₹3,000 and the event is within budget.

Output Screenshot



6. Examination Result and Grace-Mark Validation

Problem Statement

Write a Python program to classify a single examination mark as pass, grace marks required, or fail.

Assumptions

• Marks are entered as a whole number.

• Marks of 40 or more are a pass; marks from 35 through 39 are flagged as requiring grace marks; marks below 35 are a fail.

• The supplied program does not add grace marks or process multiple subjects.

Algorithm

Start.

Read exam marks.

If marks ≥ 40, display Pass.

Else if marks ≥ 35, display Grace marks required to pass.

Otherwise, display Fail.

Stop.

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input exam marks/]
    B --> C{marks >= 40?}
    C -- Yes --> D[Display Pass]
    C -- No --> E{marks >= 35?}
    E -- Yes --> F[Display grace marks required]
    E -- No --> G[Display Fail]
    D --> H([Stop])
    F --> H
    G --> H
```

Pseudocode

```
BEGIN
  INPUT marks
  IF marks >= 40 THEN
      PRINT "Pass"
  ELSE IF marks >= 35 THEN
      PRINT "Grace marks required to pass"
  ELSE
      PRINT "Fail"
  END IF
END
```

6. Examination Result and Grace-Mark Validation — Implementation

Python Source Code (as supplied)

```
marks = int(input("Enter exam marks: "))

if marks >= 40:
    print("Pass")
elif marks >= 35:
    print("Grace marks required to pass")
else:
    print("Fail")
```

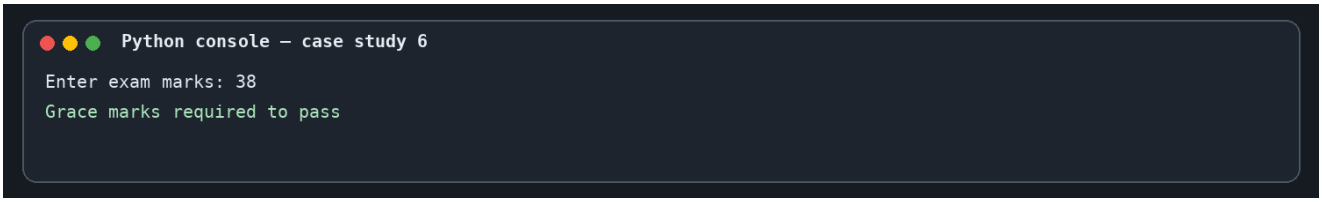
Sample Output / Console Transcript

Input values: 38

```
Enter exam marks: 38
Grace marks required to pass
```

The entered mark, 38, is below 40 but at least 35, so the program displays the grace-mark message.

Output Screenshot



7. Attendance Recovery Computation

Problem Statement

Write a Python program to calculate current attendance percentage and estimate extra classes needed to reach 75% attendance.

Assumptions

• Total classes is greater than zero and attended classes is between zero and total classes.

• The target attendance is 75%.

• The supplied formula assumes every future class is attended. Its displayed estimate uses int(needed) + 1 and may overestimate by one class when the calculated value is already an integer.

Algorithm

Start.

Read total classes and classes attended.

Calculate attendance = attended × 100 / total classes.

Display the percentage.

If attendance ≥ 75%, display sufficient.

Otherwise, estimate additional classes using (0.75 × total classes − attended) / 0.25 and display int(estimate) + 1.

Stop.

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input total classes and attended/]
    B --> C[Calculate attendance percentage]
    C --> D[/Print attendance/]
    D --> E{attendance >= 75?}
    E -- Yes --> F[Print sufficient]
    E -- No --> G[Estimate extra classes]
    G --> H[/Print estimate/]
    F --> I([Stop])
    H --> I
```

Pseudocode

```
BEGIN
  INPUT total_classes, attended
  attendance = attended * 100 / total_classes
  PRINT attendance
  IF attendance >= 75 THEN
      PRINT "Attendance is sufficient"
  ELSE
      needed = (0.75 * total_classes - attended) / 0.25
      PRINT int(needed) + 1
  END IF
END
```

7. Attendance Recovery Computation — Implementation

Python Source Code (as supplied)

```
total_classes = int(input("Enter total classes: "))
attended = int(input("Enter classes attended: "))

attendance = attended * 100 / total_classes

print("Attendance:", attendance, "%")

if attendance >= 75:
    print("Attendance is sufficient")
else:
    needed = (0.75 * total_classes - attended) / 0.25
    print("Extra classes to attend approximately:", int(needed) + 1)
```

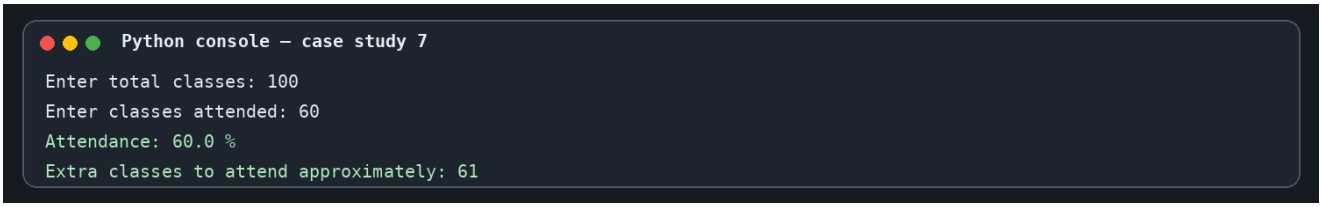
Sample Output / Console Transcript

Input values: 100, 60

```
Enter total classes: 100
Enter classes attended: 60
Attendance: 60.0 %
Extra classes to attend approximately: 61
```

Current attendance is 60%. The program estimates 61 additional classes using its implemented formula.

Output Screenshot



8. ATM Transaction Sequence Simulation

Problem Statement

Write a menu-driven Python program to check a simulated balance, deposit money, withdraw money when sufficient funds exist, and exit.

Assumptions

• The starting balance is ₹5,000.

• Menu choices are integers: 1 for balance, 2 for deposit, 3 for withdrawal, and 4 to exit.

• The supplied program checks available balance before withdrawals but does not enforce a separate withdrawal limit or reject negative transaction amounts.

Algorithm

Start with balance = 5000.

Display the menu repeatedly.

Read the user's choice.

For choice 1, display balance; for choice 2, read and add a deposit; for choice 3, read a withdrawal and subtract it only if it does not exceed balance.

For choice 4, exit the loop; otherwise, report invalid choice.

Stop.

Mermaid Flowchart Code

```
flowchart TD
    A([Start: balance = 5000]) --> B[Display menu]
    B --> C[/Input choice/]
    C --> D{Choice?}
    D -- 1 --> E[Display balance]
    D -- 2 --> F[/Input deposit; add to balance/]
    D -- 3 --> G[/Input withdrawal/]
    G --> H{amount <= balance?}
    H -- Yes --> I[Subtract amount]
    H -- No --> J[Display insufficient balance]
    D -- 4 --> K([Stop])
    D -- Other --> L[Display invalid choice]
    E --> B
    F --> B
    I --> B
    J --> B
    L --> B
```

Pseudocode

```
BEGIN
  balance = 5000
  REPEAT
    DISPLAY menu
    INPUT choice
    IF choice = 1 THEN PRINT balance
    ELSE IF choice = 2 THEN INPUT amount; balance = balance + amount
    ELSE IF choice = 3 THEN
      INPUT amount
      IF amount <= balance THEN balance = balance - amount
      ELSE PRINT "Insufficient balance"
      END IF
    ELSE IF choice = 4 THEN EXIT LOOP
    ELSE PRINT "Invalid choice"
    END IF
  FOREVER
END
```

8. ATM Transaction Sequence Simulation — Implementation

Python Source Code (as supplied)

```
balance = 5000

while True:
    print("1. Check Balance")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        print("Balance:", balance)
    elif choice == 2:
        amount = int(input("Enter deposit amount: "))
        balance += amount
        print("Amount deposited")
    elif choice == 3:
        amount = int(input("Enter withdrawal amount: "))
        if amount <= balance:
            balance -= amount
            print("Withdrawal successful")
        else:
            print("Insufficient balance")
    elif choice == 4:
        break
    else:
        print("Invalid choice")
```

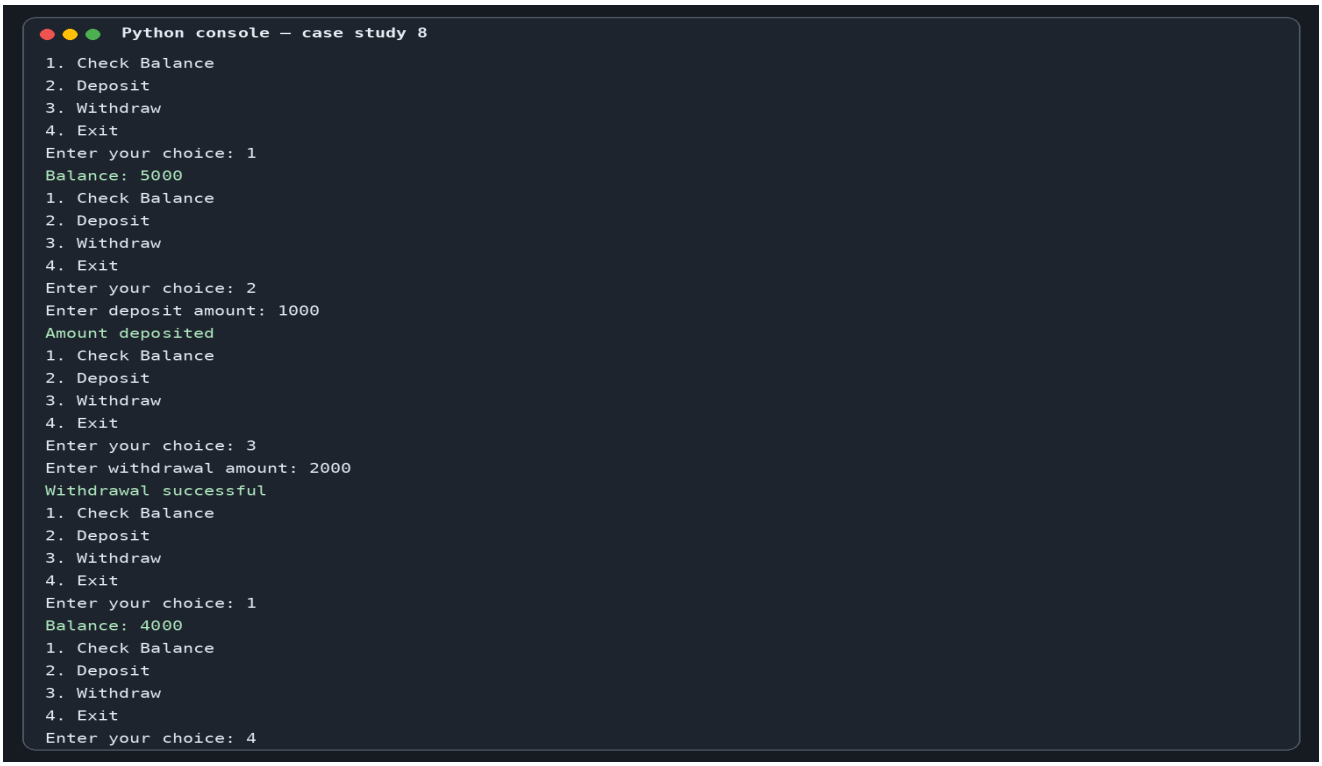
Sample Output / Console Transcript

Input values: 1, 2, 1000, 3, 2000, 1, 4

```
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 1
Balance: 5000
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 2
Enter deposit amount: 1000
Amount deposited
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 3
Enter withdrawal amount: 2000
Withdrawal successful
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 1
Balance: 4000
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 4
```

The balance starts at ₹5,000, increases to ₹6,000 after a ₹1,000 deposit, and decreases to ₹4,000 after a ₹2,000 withdrawal.

Output Screenshot


Python console — case study 8
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 1
Balance: 5000
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 2
Enter deposit amount: 1000
Amount deposited
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 3
Enter withdrawal amount: 2000
Withdrawal successful
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 1
Balance: 4000
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 4

9. Warehouse Inventory Reconciliation

Problem Statement

Write a Python program to display stock for a small list of items and update stock after a sale if enough stock is available.

Assumptions

• The initial items are Pen (50), Book (30), and Bag (20).

• The item name must match the listed capitalization exactly.

• The supplied program updates stock for one sale; it does not reconcile opening/closing stock or calculate replenishment requirements.

Algorithm

Start.

Initialize item names and stock quantities.

Display stock for each item.

Read item name and quantity sold.

If item is not in the list, display Item not found.

Otherwise, find its index and compare requested quantity with available stock.

If sufficient, subtract quantity and display updated stock; otherwise display Insufficient stock.

Stop.

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[Initialize items and stock]
    B --> C[/Display current stock/]
    C --> D[/Input item and quantity/]
    D --> E{Item exists?}
    E -- No --> F[Display item not found]
    E -- Yes --> G{Quantity <= stock?}
    G -- Yes --> H[Subtract quantity and display updated stock]
    G -- No --> I[Display insufficient stock]
    F --> J([Stop])
    H --> J
    I --> J
```

Pseudocode

```
BEGIN
  items = ["Pen", "Book", "Bag"]
  stock = [50, 30, 20]
  DISPLAY each item and its stock
  INPUT item, quantity
  IF item is in items THEN
      index = position of item
      IF quantity <= stock[index] THEN
          stock[index] = stock[index] - quantity
          PRINT updated stock
      ELSE PRINT "Insufficient stock"
      END IF
  ELSE PRINT "Item not found"
  END IF
END
```

9. Warehouse Inventory Reconciliation — Implementation

Python Source Code (as supplied)

```
items = ["Pen", "Book", "Bag"]
stock = [50, 30, 20]

for i in range(len(items)):
    print(items[i], "Stock:", stock[i])

item = input("Enter item name: ")
quantity = int(input("Enter quantity sold: "))

if item in items:
    index = items.index(item)

    if quantity <= stock[index]:
        stock[index] -= quantity
        print("Updated stock:", stock[index])
    else:
        print("Insufficient stock")
else:
    print("Item not found")
```

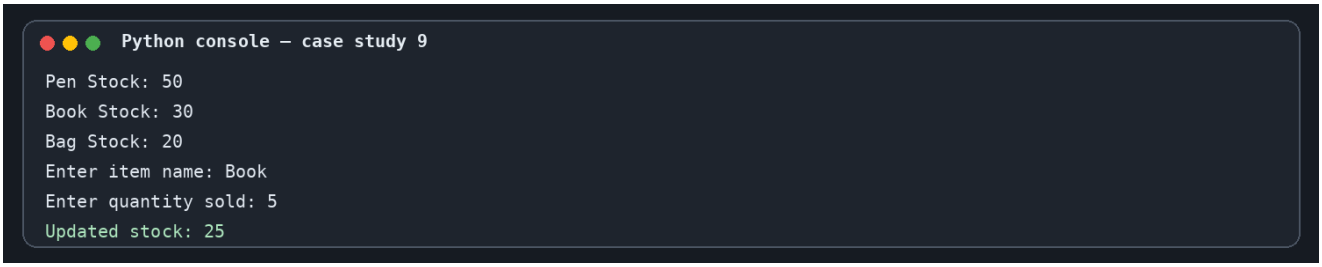
Sample Output / Console Transcript

Input values: Book, 5

```
Pen Stock: 50
Book Stock: 30
Bag Stock: 20
Enter item name: Book
Enter quantity sold: 5
Updated stock: 25
```

Book stock begins at 30. After selling 5 books, the updated stock is 25.

Output Screenshot



10. Advanced Numeric Pattern Generator

Problem Statement

Write a Python program to print a right-triangular number pattern for a user-specified number of rows.

Assumptions

• The row count is a positive integer.

• For each row i, the program prints integers from 1 through i, separated by spaces.

• The supplied program generates a right-triangular pattern, not a hollow diamond; it does not validate non-positive dimensions.

Algorithm

Start.

Read number of rows n.

For each row i from 1 through n, print numbers from 1 through i on one line.

Move to a new line after each row.

Stop.

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input number of rows n/]
    B --> C[Set i = 1]
    C --> D{i <= n?}
    D -- No --> H([Stop])
    D -- Yes --> E[Print numbers 1 through i]
    E --> F[Move to next line]
    F --> G[Increment i]
    G --> D
```

Pseudocode

```
BEGIN
  INPUT n
  FOR i = 1 TO n
      FOR j = 1 TO i
          PRINT j followed by a space
      END FOR
      PRINT newline
  END FOR
END
```

10. Advanced Numeric Pattern Generator — Implementation

Python Source Code (as supplied)

```
n = int(input("Enter number of rows: "))

for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Sample Output / Console Transcript

Input values: 5

```
Enter number of rows: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

For five rows, the program prints the numbers 1; then 1 2; then 1 2 3; continuing through 1 2 3 4 5.

Output Screenshot

